Структуры

Зачем это надо

В физическом мире нас окружают комплексные объекты. Например, Книга — это не просто абстрактная сущность. У неё есть название (строка), автор (строка), количество страниц (число) и цена (дробное число).

Без структур язык программирования видит мир слишком примитивно: только как набор отдельных чисел и букв. Структура позволяет программисту сказать компилятору: «С этого момента Book — это единый тип данных, и все эти переменные внутри него намертво связаны между собой». Составные типы данных.

Зачем это надо

- Абстракция и читаемость
- Целостность данных
- Логика размещения в памяти

Как было раньше

Если нужно было хранить данные о 50 игроках, создавали три отдельных массива: int x[50];, int y[50];, int health[50];. Чтобы изменить данные третьего игрока, нужно было вручную менять x[2], y[2] и health[2].

или

Выделялся один гигантский массив чисел, и программисты «на бумажке» считали байты. Например: «Первые 4 байта — это ID, следующие 4 байта — координата X, следующие 2 байта — здоровье». Доступ к координате X осуществлялся через указатели и ручной подсчет сдвигов в памяти (адрес начала + 4 байта). Код превращался в нечитаемое месиво

Как было раньше

Функции принимали огромные списки параметров. Написание функции для отрисовки персонажа выглядело как draw(id, x, y, h, w, color, speed, armor...) сейчас можно сделать так: draw(player)

.

Откуда они взялись

https://habr.com/ru/companies/ruvds/articles/980620/

COBOL (1959 год)

PL/I и Algol 68 (1960-е)

Язык C (1972 год)

Что такое структуры

Структура (struct) в C++ — это пользовательский составной тип данных, который объединяет несколько связанных переменных (называемых полями или членами) под одним общим именем

Синтаксис

Для определения структуры применяется ключевое слово struct, а сам формат определения выглядит следующим образом:

```
struct имя_структуры

{

    компоненты_структуры

};
```

Синтаксис

Пример

```
#include <iostream>
#include <string>
using namespace std;
struct person
{
    int age;
    string name;
};

int main(){
    person tom;
    tom.name = "Tom";
    tom.age = 34;
    cout << "Name: " << tom.name << "\tAge: " << tom.age << endl;
}
```

Синтаксис

После определения структуры мы можем ее использовать. Для начала мы можем определить объект структуры - по сути обычную переменную, которая будет представлять выше созданный тип. Также после создания переменной структуры можно обращаться к ее элементам - получать их значения или, наоборот, присваивать им новые значения. Для обращения к элементам структуры используется операция «точка»:

```
имя_переменной_структуры.имя_элемента
```

Синтаксис

Инициализация структур аналогична инициализации массивов: в фигурных скобках передаются значения для элементов структуры по порядку. Так как в структуре person первым определено свойство, которое представляет тип int - число, то в фигурных скобках вначале идет число. И так далее для всех элементов структуры по порядку.

```
person tom{ 34, "Tom" };
```

Синтаксис

Доступ к полям

Чтобы прочитать или изменить значение внутри структуры, используется оператор точка (.)

```
tom.name = «Katya"; //сменили имя на Katya

tom.age = 55; //сменили имя объекта tom
```

Синтаксис

Методы

```
struct Player {
    string nickname;
    int score;

    // Метод внутри структуры
    void printInfo() {
        cout << nickname << " набрал " << score << " очков." << endl;
    }
};

// Использование:
Player gamer {"NoobMaster", 999};

gamer.printInfo();
```

Пример

```
#include <iostream>
#include <string>

struct person
{
    person(string name, int age): name{name}, age{age}
    {
    }
    void print()
    {
        cout << "Name: " << name << "\tAge: " << age << endl;
    }
    string name;
    int age;
};

int main()
{
    person bob{"Bob", 42};
    bob.print();
}
```

Задание

- Создать структуру Cat с полями name, age, color
- Протестировать созданную структуру, создав двух объектов структуры Cat
- Создать метод Print(), выводящий информацию о коте, протестировать вызвав их у экземпляров структуры